

Welcome to Godzilla, an open-source Godzilla: Monster of Monsters framework!

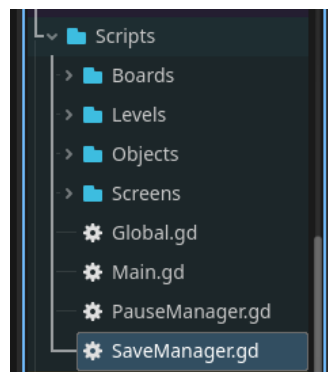
Contents

First setup	1
How the framework works: the “Main” scene	2
How the framework works: object components.....	4
How the framework works: state machine.....	6
How to create a new Pass Word Game password	7

First setup

After you open the project for the first time there’s one thing you should setup: the save system. Since there’s a save system built-in and since it uses encryption, it’s better to change the default encryption key so it’s harder for people to mess with the save files (like changing the planet or score).

The encryption key is stored in “Scripts/SaveManager.gd” file:



When you open it up and scroll to the very top, you will see a comment suggesting changing the encryption password:

```
1  extends Node
2
3  const SETTINGS_PATH = "user://settings.cfg"
4  const SAVE_FILE_PATH = "user://save.cfg"
5  const SAVE_FILE_PASS = "Godotzilla" # Change this in your game!!!
6
```

You can change it to whatever you want!

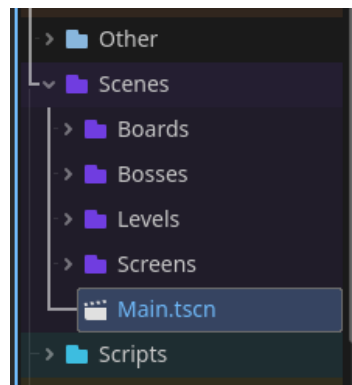
```
1  extends Node
2
3  const SETTINGS_PATH = "user://settings.cfg"
4  const SAVE_FILE_PATH = "user://save.cfg"
5  const SAVE_FILE_PASS = "My awesome game" # Change this in your game!!!
6
```

But don't panic when you change the password and all existing saves stop working. The existing save file uses the old password, and since it's now different, Godot will not be able to decrypt it.

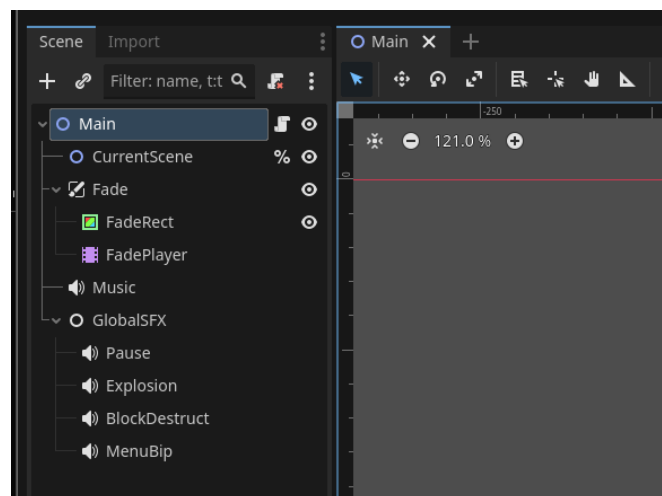
How the framework works: the “Main” scene

This section describes the basics on how the framework works. If you know how it works, you will better understand why this thing was chosen or how to better utilize that another feature, etc.

If you look in the “Scenes” folder you will see that there's a file called “Main.tscn”



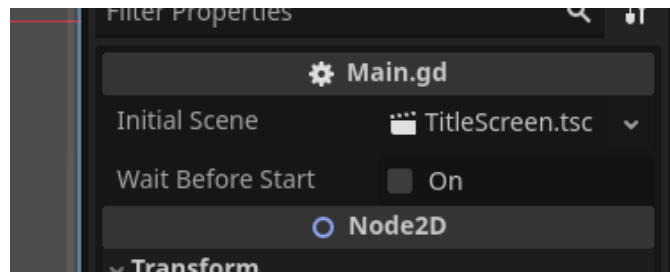
If you open it up you will see this:



It has “Music”, “Fade”, “CurrentScene” nodes and a “GlobalSFX” group of sounds. And the whole game relies on music and fades, so how did we make it so that the objects of this scene are then included into every other scene?

We didn't. This main scene includes other games scenes. Let me explain what I mean.

If you open the properties of the root node (“Main”) you will see a property called “Initial Scene”



And if you open the script tab, you will see the script for this “Main” scene. There’s a “start” function that is called when the main node is ready (“_ready” function) and it instantiates the initial scene that was passed as a property and adds it as a child of “CurrentScene” node.

```

17  > > > get_tree().process_frame.connect(_wait_before_start_func)
18  > > > else:
19  > > > start()
20  > > >
21  > > > func _wait_before_start_func() -> void:
22  > > > if not wait_before_start_flag and Global.any_action_button_pressed():
23  > > >     wait_before_start_flag = true
24  > > >     get_tree().paused = false
25  > > >     get_tree().process_frame.disconnect(_wait_before_start_func)
26  > > >     start()
27  > > >
28  > > > func start() -> void:
29  > > >     get_scene_container().add_child(initial_scene.instantiate())
30  > > >     Global.widescreen_changed.connect(_on_widescreen_change)
31  > > >     Global.scene_changed.connect(func(_from: Node, _to: Node) -> void:
32  > > >         _on_widescreen_change()
33  > > >     )
34  > > >
35  > > > Global.hide_fade()

```

Okay, but that’s just the initial scene. How do we handle scene transitions?

If you open the “Scripts/Global.gd” file, press Ctrl+F, enter “change_scene” you will be met with this:

```

109  > > > #region Scene changing
110  > > >
111  > > > ## Changes the current scene to the specified Node reference
112  > > > func change_scene_node(node: Node, free := true) -> void:
113  > > >     var curscene_parent: Node = main.get_scene_container()
114  > > >     var curscene := curscene_parent.get_child(0)
115  > > >
116  > > >     curscene_parent.remove_child(curscene)
117  > > >     if free:
118  > > >         curscene.queue_free()
119  > > >     curscene_parent.add_child(node)
120  > > >
121  > > >     scene_changed.emit(curscene, node)
122  > > >
123  > > > ## Changes the current scene to the specified scene
124  > > > func change_scene(scene: PackedScene, free := true) -> void:
125  > > >     change_scene_node(scene.instantiate(), free)
126  > > >
127  > > > ## Get the scene that the player entered upon starting the game
128  > > > func get_initial_scene() -> PackedScene:

```

change_scene 2 of 3 matches

As you can see, the “change_scene_node” gets the “CurrentScene” node in the main scene, gets its child (the initial scene that was instantiated earlier, for example), removes it and adds another node as its child. Basically the same method as the one in the main scene, but this one removes the already existing scene node that is currently being shown on screen, replaces it with another one (that was already instantiated, now it’s a node) and calls it a day.

You might be wondering, why wouldn’t we just use Godot’s built-in system for changing scenes? Like this

```
get_tree().change_scene_to_file("res://Scenes/Levels/TransitionLevel.tscn")
```

The answer is simple: it doesn’t provide enough flexibility for our needs. For example, we would manually need to instantiate our music and fade nodes. And also, Godot’s built-in system only supports changing the scene to a file, not a node.

Huh? Why would we change the scene to a node?

Here’s an example: the game created a level scene and then the player is playing the level, but then they want to pause the game and go to the pause menu. Simple enough, we just change the scene to this pause menu file.

But how would we handle exiting from the pause menu? The level was already cleared from the computer’s memory, and there was no way for us to save it anywhere.

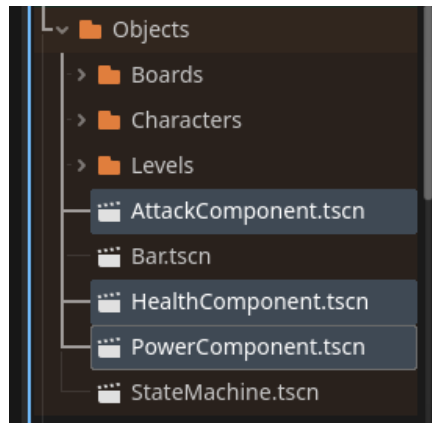
That’s where our system comes in. If you look at the above screenshot with “change_scene_node” function you can see it also takes another parameter, “free”. If it’s false, it won’t remove the previous scene node from the computer’s memory. But this parameter should only be used if you intend to save a reference to the current scene node and then get back to it some time later. Otherwise, you can just ignore this parameter.

(Don’t worry, we weren’t the first ones to introduce a system like this in place of Godot’s built-in system. You can find it in other frameworks too, such as “Sonic Worlds Next”)

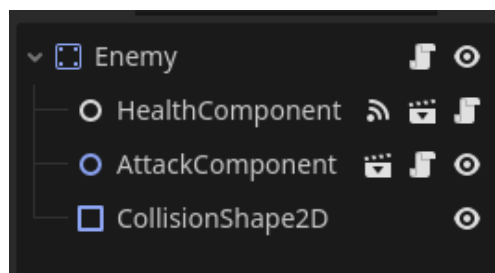
How the framework works: object components

When you create an enemy, you probably expect to every time recode the following parts: health points, attack system, or sometimes the power points for more complex enemies.

Well, you don’t have to! That’s where reusable object components come in!

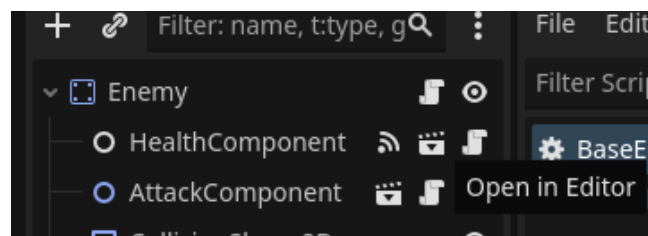


And if you open the base enemy scene (“Objects/Levels/BaseEnemy.tscn”) you can see it already has health and attack components



So how does it work and why would you want to use that instead of reprogramming these features every time?

If you open the health component scene (like this:)...



... and look into its script you can see some features that this component provides:

```

3  ## Maximum value of health the component should hold
4  @export var max_value := 10.0
5  @export var enemy := false
6  ## The amount of time in seconds the component should be invincible after getting damaged
7  @export var invincibility_time_seconds := 0.0
8  ## How many health points should be taken each frame
9  @export var health_speed := 1.0

```

```

22  ## The health amount that should be shown on screen has changed
23  signal value_changed(new_value: float)
24  signal damaged(amount: float, attack: AttackDescription)
25  signal healed(amount: float)
26  ## Happens when the maximum amount of health changes
27  signal resized(new_amount: float)
28  ## Happens when the health value reaches 0
29  signal dead
30
31  signal invincibility_started
32  signal invincibility_ended

```

And here are some other ones: it checks if its parent is hurtable and will not apply damage in this case; it makes the health points gradually go down in case a big damage was applied (for example, Godzilla's heat beam against a breakable mountain in the Wilderness level); and it already handles the invincibility time in case you need it!

```
47 func damage(attack: AttackDescription, amount := -1.0) -> void:
48     var attack_damage := amount
49     if attack != null:
50         attack_damage = attack.damage_amount
51     if attack_damage <= 0 or invincible or died \
52     or (get_parent().has_method("is_hurttable") and not get_parent().is_hurttable()):
53         return
54
55     target_value = clampf(target_value - attack_damage, 0.0, max_value)
56     damaged.emit(attack_damage, attack)
57     if invincibility_time_seconds > 0.0:
58         invincible = true
59         invincibility_started.emit()
60
61         await get_tree().create_timer(invincibility_time_seconds, false).timeout
62
63         invincible = false
64         invincibility_ended.emit()
```

The attack component includes an Area2D node, and if the component in its attacking state and another body that has a health component enters the area2d node, it applies damage to the node and informs the parent that a body was attacked (for example, that's how it's used for the player:)

```
311 func _on_attack_component_attacked(attacked_body: Node2D, attack: AttackDescription) -> void:
312     if attacked_body is Enemy:
313         add_xp(5)
314         Global.add_score(int(20 * attack.damage_amount))
```

And the last one, the power component. It handles the power amount, and it provides a check if the current power amount is enough for an object's attack or any other action.

How the framework works: state machine

When you're just starting to code a game, your player script might be simple enough to include every action you currently have. But when your game grows, it becomes harder to manage every action if all of them are in one single file.

That's where state machine comes in. It separates actions into different nodes that are enabled or disabled when the action is changed. You can see some player actions code in "Scripts/Objects/Characters/States" folder, and in "Scripts/Objects/StateMachine.gd" file you can see how a reusable state machine was made.

How to create a new Pass Word Game password

Here's a pretty simple tutorial that is too small to be included in a separate file. If you open the "Scripts/Screens/PassWordGame.gd" file and scroll up you will see the list of every password that is possible to enter in this mode:

```
26  ▾ ## Dictionary of all passwords.
27  ## Passwords should be lowercase and stripped from spaces on the right
28  ▾ var passwords: Dictionary[String, Callable] = {
29    ▸ "test": pw_test,
30  }
```

Let's create a new one.

```
28  ▾ var passwords: Dictionary[String, Callable] = {
29    ▸ "test": pw_test,
30    ▸ "g0j1ra": pw_test,
31  }
```

Remember that not every letter is included in the pass word game alphabet!

Now if you scroll all the way down to the bottom of the script you will find the "pw_test" function:

```
178  ▾ #####
179  # Password actions below #|
180  #####
181
182  ## Just a test password
183  ▾ func pw_test() -> void:
184    ▸ get_tree().paused = true
185    ▸
186    ▸ Global.music_fade_out()
187    ▸ await Global.fade_out(Global.FadeColor.WHITE)
188    ▸ await get_tree().create_timer(1).timeout
189    ▸
190    ▸ Global.play_music(music)
191    ▸ await Global.fade_in(Global.FadeColor.WHITE)
192    ▸
193    ▸ get_tree().paused = false
```

Let's create a similar one but instead it will just play the credits music:

```
195  ▾ func pw_g0j1ra() -> void:
196    ▸ Global.play_music(preload("res://Audio/Soundtrack/Screens/Credits.ogg"))
```

Good, now we will reference it in the passwords list

```
28  ▾ var passwords: Dictionary[String, Callable] = {
29    ▸ "test": pw_test,
30    ▸ "g0j1ra": pw_g0j1ra,
31  }
```

Now you can test the game and verify that it works!

